A FRAMING

The Intentional Programming Paradigm

You don't write the program. You express intent — in any medium — and converge on the result through an iterative loop with a reasoning agent. Declarative in spirit, but with natural language as the spec and dialogue as the compiler.

People call this "chat programming" or "vibe coding" — but those name the interface and the vibe, not the paradigm underneath. What's actually new is a shift along the oldest axis in programming — how much of the "how" you have to specify yourself — plus something no earlier paradigm had: the work spans many languages at once, runs on many minds at once, and teaches you while you do it.

The lineage of paradigms

Every paradigm shift has moved the programmer further from the machine's "how" and closer to pure "what." This is simply the newest link in that chain.

1940S–50S · MACHINE & ASSEMBLY

Imperative, at the hardware

You specify **every instruction and register**. The machine does exactly and only what you say. Maximum control, maximum burden.

1957+ · PROCEDURAL / STRUCTURED

Fortran, COBOL, C

You specify **the steps**, but in human-readable form. A compiler translates to machine code — the first big delegation of "how."

1958+ · FUNCTIONAL / DECLARATIVE

Lisp, SQL, Prolog

You specify **the what — the target result**. A fixed engine (query planner, resolver) computes the steps. Deterministic: a correct spec yields a correct result in one shot.

1967+ · OBJECT-ORIENTED

Simula, Smalltalk, C++, Java

You specify **objects and their interactions** — organizing complexity around state and behavior rather than raw procedure.

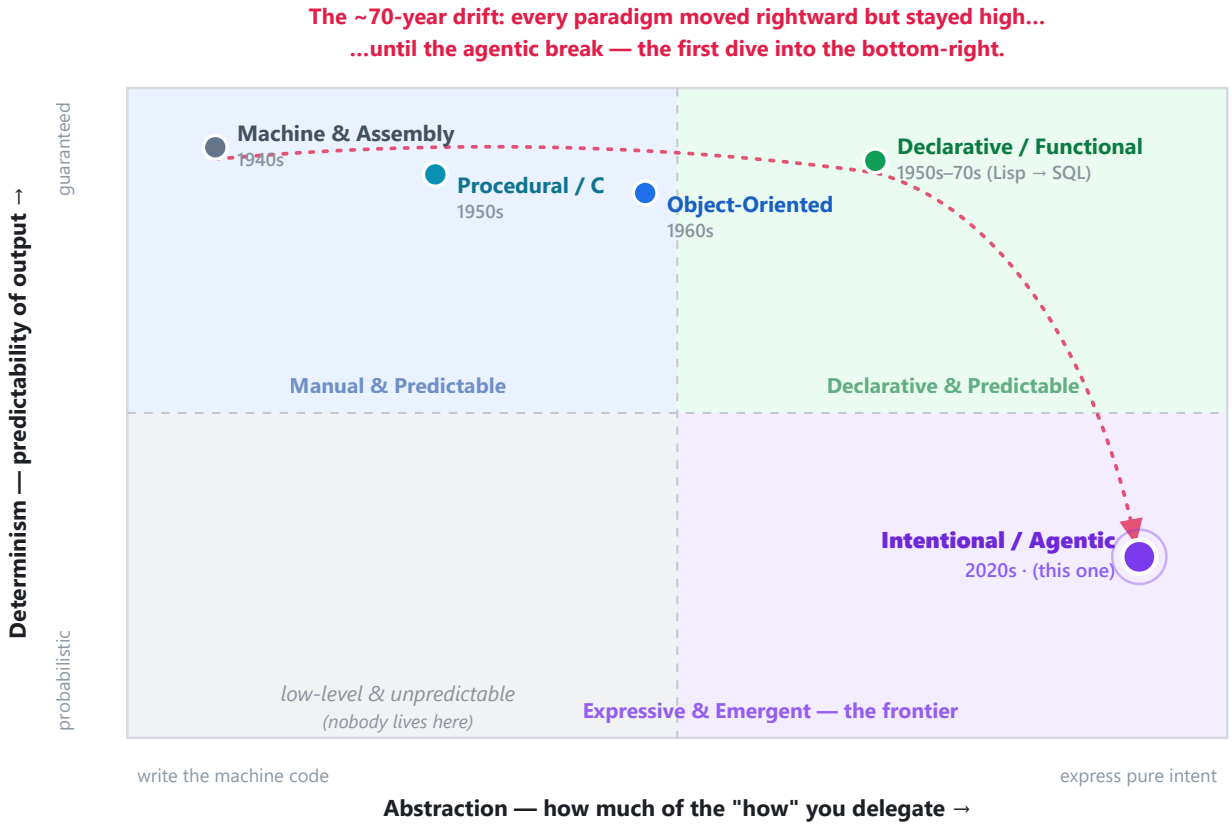
2022+ · INTENTIONAL / AGENTIC THIS ONE

Multimodal intent + a reasoning harness

You specify **intent, in whatever medium fits**. A *reasoning* engine infers the how — across many languages at once — and you converge on correctness through a feedback loop. The compiler is a conversation.

The paradigm quadrant

Plot the paradigms on the two axes that carry this whole story — **abstraction** (how much of the "how" you delegate) against **determinism** (how predictable the output is) — and the history draws itself.



Machine & Assembly · 1940s Procedural · 1950s Object-Oriented · 1960s Declarative · 1950s–70s

Intentional / Agentic · 2020s

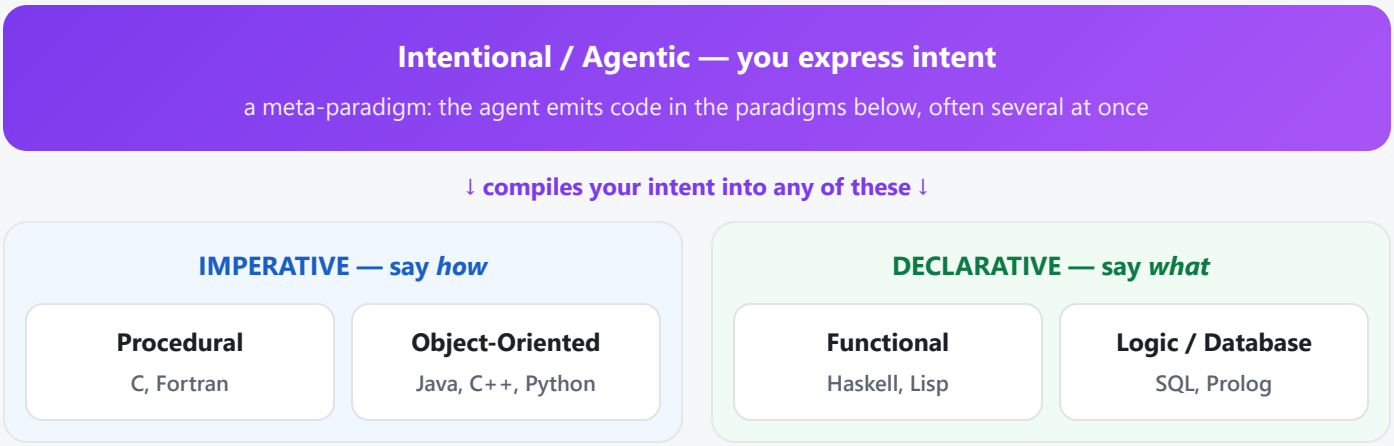
Decade = when the paradigm first became operational in practice.

What the chart says. For roughly seventy years, every paradigm drifted in the *same* direction — rightward, delegating more of the "how" — while never leaving the high-determinism band across the top. SQL is far more abstract than assembly, yet both do exactly and only what you specify. That's the through-line: more abstraction, same predictability.

Intentional programming is the first genuine break in the pattern. It lunges further right than anything before it *and* drops out of the deterministic band into the bottom-right — "expressive and emergent." It bought a historic leap in abstraction and expressiveness by **spending determinism**. Notice the bottom-left stays empty: nobody wants a language that is both low-level *and* unpredictable — the entire point of surrendering determinism is to buy abstraction in return. This reframes the loop, the tests, and the harness not as nice-to-haves but as **the toll you pay to operate safely in the bottom-right quadrant.**

What a paradigm is — and where this one sits

Before slotting intentional programming into the family, it helps to be precise about what a paradigm actually is. A paradigm is **not a language or a tool** — you can't build anything with one. It's a set of ideas and conventions about how to structure a program. And paradigms are **not mutually exclusive**: most modern languages are multi-paradigm — Python and JavaScript happily mix object-oriented, functional, and declarative styles in the same file. Classically these styles form a family tree with two great branches — *imperative* (say *how*) and *declarative* (say *what*):



Here's where intentional programming fits — and it isn't simply a sixth leaf on the tree. Because you specify intent and a reasoning engine chooses the implementation, it sits *above* the tree as a kind of **meta-paradigm**: from one natural-language request the agent can emit procedural, object-oriented, functional, or declarative code — and routinely blends them in a single artifact. (That's exactly what the guess-a-number task below shows: one description, rendered five different ways.) It doesn't replace the older paradigms; it *orchestrates* them.

This also explains why it feels like the natural endpoint of a decades-long trend. Declarative programming was always described as "hiding away complexity and bringing programming closer to human thinking" — you order the dish and let the chef decide how to cook it, in the familiar restaurant analogy. Intentional programming pushes that to its limit: the "what" is now plain human language, and essentially *all* of the "how" — across every layer and paradigm — is hidden. And echoing the old wisdom that there is no single best paradigm, the right move is still to fit the paradigm to the problem; the difference is that now you can ask for it in words instead of hand-coding it.

From instructing to steering

The clearest way to see the shift is to place it on that same axis. Imperative programming makes you specify the *how*, step by step. Declarative programming lets you specify the *what* and hands the *how* to a fixed engine. Intentional programming lets you specify intent in natural language, hands the *how* to a *reasoning* engine, and then — crucially — lets you and the engine converge on the result together.

So the programmer's job changes shape. Less "how do I implement this," more "what exactly do I want, and is this it?" The scarce skills become: **describing intent precisely, recognizing when a result is right, and knowing which correction moves the system toward the goal.** You stop being the person who types the solution and become the person who defines and steers it.

What actually makes it new

Three properties separate this from classical declarative programming. They matter because two are strengths and one is the weakness the whole workflow is built to manage.

Property 1

Multimodal intent

The spec isn't restricted to text. A screenshot, a voice memo, a sketch, an error log, or an example are all valid ways to express what you want. "Chat" undersells this — the point is *intent in the most natural form*.

Property 2 · the risk

The "how" is inferred

Unlike a deterministic query planner, the engine *reasons*. The mapping from intent to implementation is probabilistic — sometimes better than you'd have specified, sometimes wrong. A genuinely different relationship to the machine.

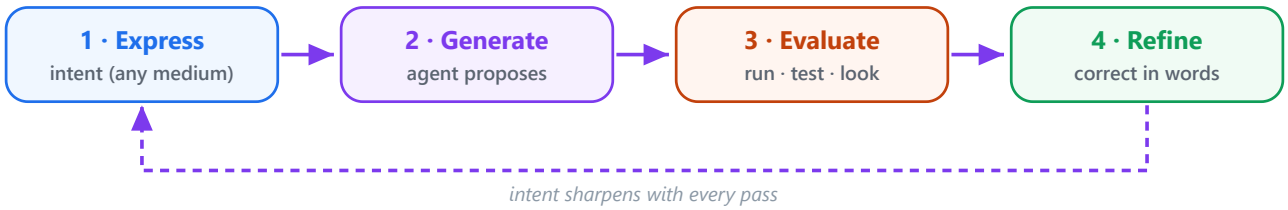
Property 3 · the fix

Correctness by convergence

You rarely specify perfectly upfront. You describe → observe → test → correct → repeat. The spec and the artifact *co-evolve*. This is what compensates for the non-determinism of Property 2.

The loop is the paradigm

Reframe the act of programming from a linear "write instructions" into a four-move cycle. The programmer's job shifts from *authoring instructions* to *articulating intent and judging outcomes*.



What the paradigm uniquely unlocks

Beyond the loop itself, four capabilities appear that no earlier paradigm offered together. These are the parts that feel like a genuine change in kind, not just degree.

UNLOCK 1 · POLYGLOT BY DEFAULT

One intent, many tongues

A single expressed demand can fan out across the entire stack at once — Python for the logic, SQL for the query, HTML and CSS for the surface, a REST call to a third-party API, a shell script to wire it together. You ask one question; five languages and three services answer it, already stitched into a working whole. The polyglot tax — the mental cost of constantly re-compiling your brain into another syntax and swallowing the friction between layers — is absorbed by the agent.

Python | SQL | HTML | CSS | REST API | shell

> ANALOGY · THE FILM DIRECTOR
>
> A director says "make this scene feel tense and rainy at dusk." The cinematographer, gaffer, colorist, sound designer, and composer each translate that one sentence into their own technical craft — simultaneously — and it comes out as a single coherent scene. You never learned to grade color or rig a boom mic. You expressed the vision; the departments spoke their own languages. Intent programming is directing: one demand, many trades, one coherent cut.

UNLOCK 2 · DISTRIBUTED COGNITION

Not one mind — a swarm

Classical programming is bottlenecked by a single human working memory: one person, one problem in focus, one mental stack at a time. Intentional programming attacks the challenge with *many contexts and agents at once* — one drafting the schema while another reasons about the UI while another checks the edge cases. The cognition is parallel and distributed, and you sit at the head of the table synthesizing, not carrying every thread yourself.

> ANALOGY · THE SITUATION ROOM & THE ORCHESTRA
>
> Instead of one analyst working a problem sequentially, imagine a war room where specialists hit it from every angle at the same time — and you're the one at the head of the table deciding which threads to pull. Or an orchestra: one score, many instruments sounding together. You conduct; you don't play every part. The lone programmer was a soloist. Here you've become a conductor.

UNLOCK 3 · IT TEACHES YOU BACK

The loop that makes *you* smarter

The mechanism doesn't only produce something new and useful — it teaches the person steering it. Because the agent often solves a problem in a way you wouldn't have, every pass exposes you to unfamiliar libraries, patterns, and techniques. You walk away from a build knowing more than you did, with a wider sense of how a problem *could* be solved. The artifact is the output; the education is the byproduct — and often the more durable one.

> ANALOGY · THE MASTER CRAFTSMAN NARRATING
>
> Copy-pasting a fix from a forum hands you a patch and teaches you nothing. Working the loop is more like apprenticing beside a senior who narrates every decision as they make it: you get the deliverable *and* the reasoning. It's a flywheel — better questions produce better answers, which teach you to ask better questions. The paradigm is generative of skill, not just of software.

UNLOCK 4 · MANY HUMAN TONGUES

Any language — and the culture inside it

Intent isn't tied to English. You can express what you want in French, German, Japanese, Arabic — or blend several in the same prompt. Beyond sheer convenience, this carries a subtler benefit: every language encodes its own idioms, defaults, and cultural framings, so the tongue you think in can nudge both the solution and its tone. Describing a problem in your mother tongue often sharpens the intent itself, and asking across languages lets you pull in more than one cultural perspective at once.

> **ANALOGY · THE MULTILINGUAL DESIGN TEAM**
>
> It's like briefing a team where each member thinks in a different language and brings their own instincts. Ask in English and you get one set of defaults; ask in Japanese and the politeness and formality shift; describe it in your own language and you simply express yourself more precisely. Same brief, a richer set of perspectives.

Harnessing the loop: the hidden variable

Here's what "chat programming" conceals entirely: **output quality is not a property of the paradigm — it's a property of the model and the harness around it.** The harness is the scaffolding that turns a one-shot text generator into a converging loop. It gives the model *eyes* (read errors, run tests, see screenshots), *hands* (edit files, execute commands), and *discipline* (plan, verify, retry).

This is why the same model feels like a different tool depending on where you run it. In 2026 this became its own discipline — *harness engineering* — after a widely-cited teardown estimated roughly **98% of a leading coding agent is the harness, not the model**. The model is the engine; the harness is the entire car.

```
# the two things chat programming hides
quality ≈ f( intent clarity × model capability × harness quality × iteration
richness )

cost    ≈ tokens per turn × number of turns × model price
# stronger model + richer loop → more $ and latency, fewer turns to converge
```

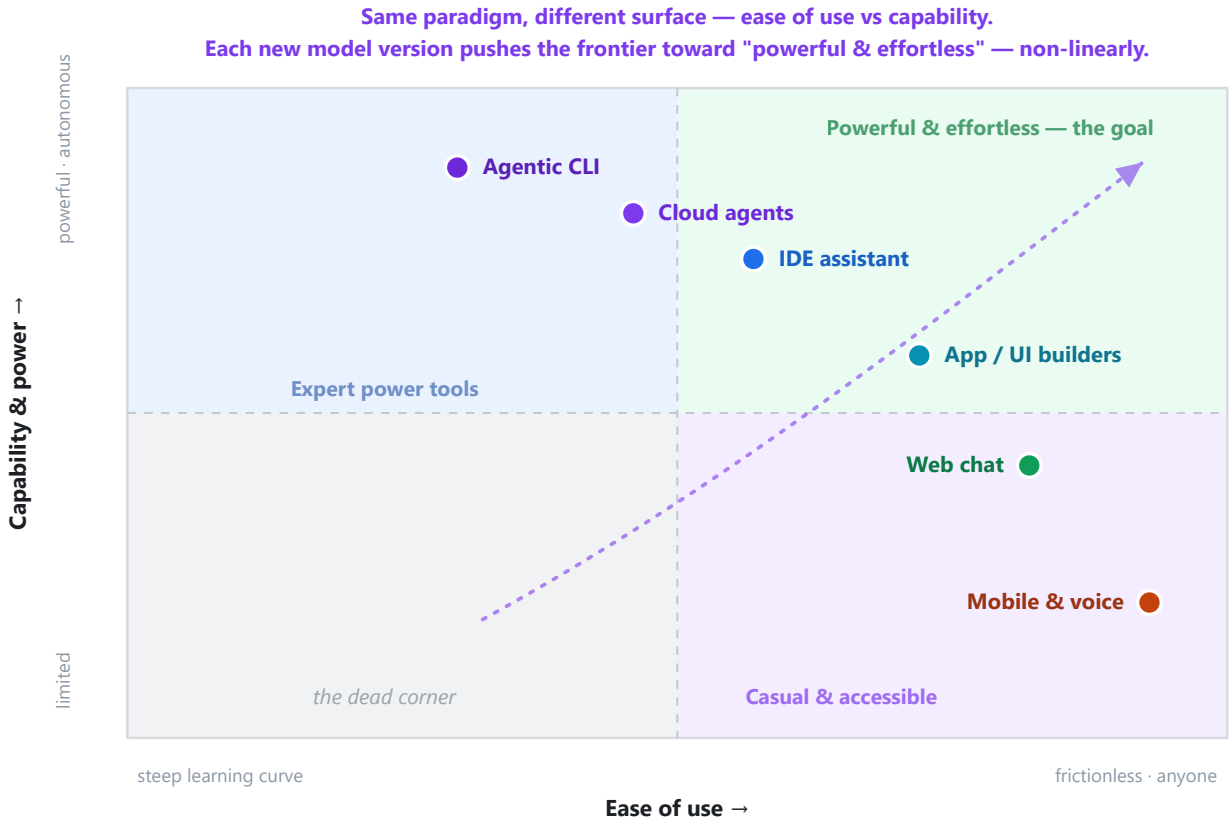
The quality / cost / latency triangle. Because the "how" is inferred and correctness is reached by iteration, you're always picking a point on a triangle. Cheap one-shots are cheap precisely because they skip the loop. A strong model in a deep harness costs more per turn and is slower — but converges in fewer turns and reaches places a weak setup never will. There is no universal "best"; there's only the right point for the stakes of the task.

One more thing the popular name hides: the probabilistic core — the reasoning engine behind Property 2 — is an **LLM, and every LLM is the product of a particular lab** that trained and aligned it. Claude comes from Anthropic, GPT and Codex from OpenAI, Gemini from Google, and so on. "Choosing the model" is therefore also choosing a lab's judgment about how it should reason, refuse, and behave — so two harnesses running different labs' models can answer the very same intent quite differently.

Which leads to the blunt, practical truth: **the same paradigm and the same prompt — run through a different IDE, CLI, or harness, or backed by a different model — can produce vastly different quality.** The paradigm is constant; the results are not. So it helps to see the landscape of surfaces you can actually run this on.

The tooling quadrant: ease vs capability

Map the surfaces by **ease of use** against **raw capability**, and note how each one reaches you across devices — phone, cloud, desktop, wearable.



D · Desktop C · Cloud P · Phone W · Wearable

SURFACE	EXAMPLES	RUNS ON
Agentic CLI	Claude Code, Codex CLI, Gemini CLI	D C
Cloud / async agents	background coding agents	C D P
IDE assistant	Cursor, Copilot, Windsurf	D
App / UI builders	v0, Bolt, Lovable, Replit	D C
Web chat	Claude.ai, ChatGPT, Gemini	D P C
Mobile & voice / wearable	phone apps, voice assistants	P W C

And none of this sits still. Each new model version — usually the latest — **resets the ceiling**, adding capabilities that didn't exist a release ago. The frontier isn't advancing in a straight line; it lurches outward in jumps, which is why a workflow that felt impossible last quarter can be routine this one. Treat the quadrant as a snapshot of a frontier in motion, drifting steadily toward "powerful *and* effortless."

The honest tension

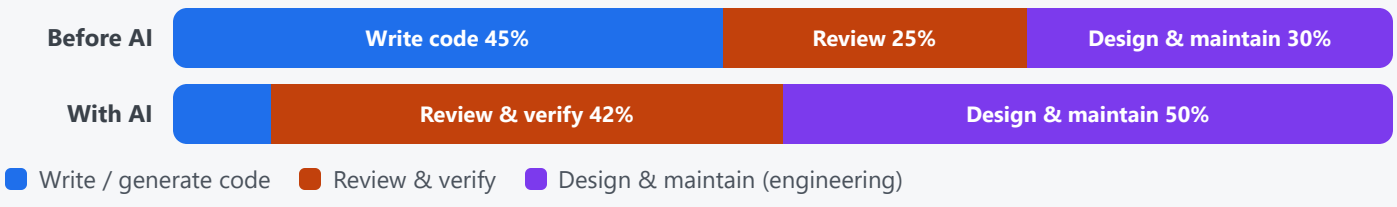
There's a real cost baked into all of this, and it's worth naming plainly. Because the "how" is inferred and correctness is reached by convergence, you trade **precision and determinism** for **expressiveness and speed**.

> Declarative code does *exactly and only* what it says. Intentional code does *approximately what you meant* — until you've steered it into place. That's why testing isn't decoration in this paradigm; it's structural. **Testing is the mechanism that substitutes for the determinism you gave up.** The loop and the tests are not add-ons to intentional programming — they are the thing that makes it safe to trust at all. (It's the toll for the bottom-right quadrant.)

Programming vs. software engineering — and where the speed goes

A distinction worth drawing sharply, because the paradigm hits the two halves very differently. **Programming** is solving a problem by writing code and running it. **Software engineering** is collaborating to design and maintain a durable system that keeps evolving long after the first author has moved on. Programming is *part* of software engineering — but only a part. (The framing is Google's, from their August 2026 argument that language choice matters *more* in the AI era, not less.)

Here's the twist. Intentional programming collapses the cost of *programming* — the writing — to almost nothing: a capable agent emits hundreds of lines of valid code in seconds. But it barely touches *software engineering*. So the bottleneck of the whole lifecycle moves: from generation to verification. For decades a language's productivity was measured by how easy it was to write. Now that writing is nearly free, what matters is how easy the result is to **read, review, verify, and maintain.**



Illustrative. AI shrinks writing to a sliver; the human bottleneck shifts to reviewing, verifying, and the broader engineering AI can't own.

This is why "fast with AI" is a mirage if you measure it at the keyboard. The generation step got orders of magnitude faster; the human review-and-verify step did not. Speed at the wrong stage just piles unreviewed code against the real constraint. Worse, autonomous agents can open hundreds of pull requests and refactor whole services on a whim — so the danger isn't slow typing, it's **architectural drift** accumulating faster than anyone can review it.

> **AI is a teammate — a talented maverick that needs guardrails.** It writes fast but has a narrow view of the whole system, so humans still own the parts that are pure software engineering: the architecture, the service boundaries, the reliability and security of production. And the guardrails that make the maverick safe are exactly the deterministic tools — static types, a compiler, formatters, tests, fuzzing — that let the agent *self-correct before a human ever looks*. Static types plus a fast compile give the tightest iteration loop; tellingly, the languages that were hardest for humans (strict, verbose, compiled) turn out to be the easiest for agents.

Notice where this lands on the quadrant. The agentic paradigm dove into the low-determinism corner; these guardrails are precisely how you claw determinism back — the toll for living in the bottom-right. Which yields a genuinely counterintuitive conclusion: **as you write less code, your choice of language and tooling matters more, not less** — because that choice now governs the review, verification, and maintenance surface, not the typing surface. Google's case is that Go fits unusually well (read-first clarity, static types, one canonical format, fast compile, never-break compatibility); the deeper, language-agnostic point is that *deterministic, uniform, well-tooled* stacks absorb high-velocity agent output best.

Who's actually good at this

There's no single "best person" — but there's a clear dividing line, and it maps exactly onto Property 3. The people who ship real software with this paradigm are the ones who *master the loop and the harness*. The people producing throwaway demos are the ones who skip it.

AK

Andrej Karpathy NAMED IT

Coined "vibe coding" (Feb 2025) and, earlier, "the hottest new programming language is English." Crucially, he framed pure vibe coding — "*I Accept All always, I don't read the diffs*" — as fit for "*throwaway weekend projects,*" not production. He named the extreme, not the discipline.

SW

Simon Willison THE DISCIPLINE

Drew the defining line: if an LLM wrote every line but you *reviewed, tested, and understood it*, "that's not vibe coding — that's using an LLM as a typing assistant." His 2026 term **"vibe engineering"** is this paradigm done properly: agents inside a real process with tests, review, source control, and planning.

KV

Kenton Varda / Cloudflare RECEIPTS

Built a production OAuth library largely through an AI agent and *committed the prompts alongside the code* — a public, auditable example of the loop used with engineering rigor rather than blind acceptance.

PL

Pieter Levels (@levelsio) VELOCITY

The archetype of shipping and monetizing fast with AI (e.g. a browser flight-sim that went viral). Proof the paradigm's speed is real — and, in the discourse, a lightning rod for the fragility of skipping the loop on anything that has to last.

Named examples are illustrative and drawn from public writing through 2025–2026; the field moves fast, so treat specifics as a snapshot, not a leaderboard.

Why the YOLO approach misses the point

"YOLO" — one-shot the prompt, accept everything, don't test, don't steer — looks like the purest expression of the paradigm. It's actually the opposite. It throws away the one thing that makes the paradigm work.

The core problem: Property 2 (the "how" is inferred, so output is non-deterministic) is a liability. Property 3 (convergence through the loop) is the *only* thing that pays it off. YOLO keeps Property 2 and discards Property 3 — so you get maximum exposure to the weakness with none of the mitigation. On the quadrant, it's trying to live in the bottom-right without paying the toll.

Without evaluation and refinement you're not really using a reasoning loop at all. You're back to *hoping a probabilistic generator got it right the first time* — which reintroduces the determinism problem of the old world without inheriting any of its determinism. Worse, it forfeits Unlock 3 entirely: skip the loop and you learn nothing from the build. That's why

Karpathy scoped it to throwaway projects and why Willison renames the serious version "vibe engineering." The value was never in the generating. It was always in the loop.

One task, every paradigm: guess a number (1–10)

The same tiny program — pick a secret number from 1 to 10, keep guessing until you hit it — written the way each paradigm would. Watch how much "how" you shed moving down the list, until the last one, where the "source code" is simply intent in plain human language.

Machine & Assembly

IMPERATIVE · YOU MANAGE REGISTERS

```
; x86-64 (NASM) — core compare loop; RNG + I/O syscalls elided
; rbx = secret (1..10)      rax = the player's guess
guess_loop:
    call    read_guess        ; returns guess in rax
    cmp     rax, rbx
    je      .win              ; guess == secret
    jl      .too_low          ; guess < secret
    mov     rdi, msg_high     ; else guess > secret
    call    print
    jmp     guess_loop
.too_low:
    mov     rdi, msg_low
    call    print
    jmp     guess_loop
.win:
    mov     rdi, msg_win
    call    print
    ret
```

You steer registers and branches by hand; even reading a number is a syscall you wire yourself. (I/O and RNG abridged — in full it's ~80 lines.)

Procedural / C

IMPERATIVE · READABLE STEPS

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));
```

```
    int secret = rand() % 10 + 1, guess;
    do {
        printf("Guess (1-10): ");
        scanf("%d", &guess);
        if (guess > secret)      puts("Too high");
        else if (guess < secret) puts("Too low");
    } while (guess != secret);
    puts("Correct!");
}
```

Still every step by hand, but in human-readable form — a compiler handles the machine code.

Object-Oriented / Python

STATE WRAPPED IN AN OBJECT

```
import random

class GuessGame:
    def __init__(self, lo=1, hi=10):
        self.secret = random.randint(lo, hi)

    def guess(self, n):
        if n == self.secret: return "correct"
        return "high" if n > self.secret else "low"

game = GuessGame()
while True:
    result = game.guess(int(input("Guess (1-10): ")))
    if result == "correct":
        print("Correct!"); break
    print("Too", result)
```

The secret becomes state inside an object; the guessing behavior hangs off it.

Functional / Haskell

RECURSION · PATTERN MATCHING

```
import System.Random (randomRIO)

main :: IO ()
main = randomRIO (1, 10) >>= play

play :: Int -> IO ()
play secret = do
  putStr "Guess (1-10): "
  guess <- readLn
  case compare guess secret of
    EQ -> putStrLn "Correct!"
```

```
    GT -> putStrLn "Too high" >> play secret
    LT -> putStrLn "Too low"  >> play secret
```

No mutable loop — the game recurses, and matching on `compare` replaces the explicit branches.

Declarative / SQL

DESCRIBE THE RESULT · ENGINE COMPUTES

```
-- 1) The rule as a query: for any secret + guess, declare the verdict
SELECT CASE
         WHEN guess = secret THEN 'correct'
         WHEN guess > secret THEN 'too high'
         ELSE 'too low'
       END AS verdict
FROM (SELECT 7 AS secret, 5 AS guess) AS t;      -- -> 'too low'

-- 2) Play a whole game declaratively: a recursive CTE binary-searches 1..10
--    (integer division assumed — e.g. Postgres / SQLite / SQL Server)
WITH RECURSIVE play(lo, hi, guess, secret, step) AS (
    SELECT 1, 10, (1 + 10) / 2, 7, 1             -- secret is 7 (any 1..10)
  UNION ALL
    SELECT CASE WHEN guess < secret THEN guess + 1 ELSE lo END,
           CASE WHEN guess > secret THEN guess - 1 ELSE hi END,
           CASE WHEN guess < secret THEN (guess + 1 + hi) / 2
                WHEN guess > secret THEN (lo + guess - 1) / 2 END,
           secret, step + 1
    FROM play
    WHERE guess <> secret
)
SELECT step, guess,
       CASE WHEN guess = secret THEN 'correct'
            WHEN guess > secret THEN 'too high'
            ELSE 'too low' END AS verdict
FROM play ORDER BY step;
-- step | guess | verdict
--   1  |   5   | too low
--   2  |   8   | too high
--   3  |   6   | too low
--   4  |   7   | correct
```

Possible — but with an honest limit, and the limit is the point. Pure SQL describes a *result set*, not an interactive session: it has no built-in way to prompt you, wait, read a guess, and loop. So the *human-guessing* loop isn't expressible in pure declarative SQL — that needs the application layer to feed guesses, or a procedural extension (PL/pgSQL, T-SQL) with real loops, which is no longer purely declarative. What SQL *does* do beautifully is declare the game's logic: query 1 states the verdict rule for any secret/guess pair, and the recursive CTE in query 2 plays an entire game by binary search — you never write a loop; you declare how each state follows from the last

and the engine iterates to the answer. That is the declarative spirit exactly: say *what* the result is and let the engine find the *how*.

Intentional / Agentic (vibe coding)

NATURAL LANGUAGE · INTENT IS THE SOURCE

No syntax to write — you describe the outcome and steer. And the intent needn't be in English; use any human language, or several at once:

ENGLISH

Build a tiny game: pick a secret number from 1 to 10, let me guess over and over, tell me "too high" or "too low" each time, and congratulate me when I get it.

FRANÇAIS

Crée un petit jeu : choisis un nombre secret entre 1 et 10, laisse-moi deviner, dis-moi « trop grand » ou « trop petit » à chaque essai, puis félicite-moi quand je trouve.

DEUTSCH

Bau ein kleines Spiel: Wähle eine geheime Zahl von 1 bis 10, lass mich immer wieder raten, sag „zu hoch" oder „zu niedrig", und gratuliere mir, wenn ich sie errate.

Each of these yields the same working app — in whatever stack you ask for (a Python script, a web page, a phone app), with none of the syntax above coming from you. The language you choose also colors the output: French and German nudge different politeness and phrasing defaults in the messages, and stating the problem in your mother tongue tends to sharpen the intent itself. Blend languages in one prompt to pull in several cultural perspectives at once.

How this was made: this entire document — the text, the charts, the code samples, and the playable game below — was generated in **Claude Desktop using Opus 4.8**, steered through exactly the loop it describes.

...and the app it produced: TypeScript

LIVE · RUNS IN YOUR BROWSER

```
type Result = "correct" | "high" | "low";

class GuessGame {
  private secret: number;
  constructor(lo = 1, hi = 10) {
    this.secret = Math.floor(Math.random() * (hi - lo + 1)) + lo;
  }
  guess(n: number): Result {
    if (n === this.secret) return "correct";
    return n > this.secret ? "high" : "low";
  }
}
```

```
const game = new GuessGame();
// wire game.guess(n) to the input below and show the result
```

1–10 Guess New game

I'm thinking of a number between 1 and 10. Take a guess!

The block above is the TypeScript source; the widget is that exact logic (transpiled to JavaScript) running live. Same game as the four versions before it — but this one you steered into existence by describing it, not by writing the loop.

The paradigm, in memes

Sticky lines for explaining the novelty and the benefits — the kind that survive a hallway conversation.

Describe the destination, not the route.
You express intent; the agent infers the how.

The compiler is a conversation.
Natural language in, working software out — by dialogue.

One prompt in, five languages out.
Python, SQL, HTML, CSS, a REST call — from one demand.

You're the director, not the crew.
Cast the vision; the departments speak their own craft.

Not one mind — a swarm.
Many contexts hit the problem in parallel; you conduct.

Build something, learn something.
Every pass through the loop levels you up.

The model's the engine. The harness is the car.

YOLO keeps the risk and drops the loop.

Same model, different harness, wildly different results.

No test, no steer — just hoping the dice land right.

AI programs. You engineer.

Generation is cheap; design, review, and upkeep are the job.

Writing got cheap. Reviewing is the bottleneck.

The constraint moved from typing to verifying.

In one sentence: Intentional programming is a paradigm where you specify software through *multimodal intent* and converge on the result through an *iterative feedback loop* with a reasoning agent — one demand fanning out across many languages and many parallel minds, where the model sets the ceiling, the harness closes the loop, the loop teaches you as it builds, and skipping the loop (YOLO) discards the very mechanism that makes the whole thing trustworthy.

Sources & further reading

This framing synthesizes a few public pieces — orientation guides to the classic paradigms, plus recent arguments about what AI changes:

- **Google Developers Blog** — Why Go is an Ideal Language for AI-Assisted Software Engineering (Aug 2026): the shift of the bottleneck from writing to verification, and the programming-vs-software-engineering distinction.
- **freeCodeCamp** — Programming Paradigms – Paradigm Examples for Beginners (German Cocca): what a paradigm is and isn't, and declarative programming as "hiding the how."
- **DataCamp** — An Introductory Guide to Different Programming Paradigms: the restaurant/chef analogy and the "no single best paradigm" point.
- **DEV Community** — The World of Programming Paradigms (Favourite Jome): the imperative-vs-declarative family tree.
- Plus **Andrej Karpathy** (who coined "vibe coding," Feb 2025) and **Simon Willison** (who drew the "vibe engineering" line) for the practitioner framing used in the sections above.

A structured reflection · framing of the agentic / intentional programming paradigm · generated in Claude Desktop with Opus 4.8